Performance Testing

Date	15 March 2026
Team ID	xxxxxx
Project Name	**A Comprehensive Measure of Well-Being**
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Postman, Flask Local Server
Type of Testing	Load Testing, Functional Testing
Target Module / API	/predict endpoint (HDI Prediction API)
Test Environment	Local System (Windows/Mac, Python 3.12, Flask)
Test Date	15 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Single user submits HDI indicators	1	30	Prediction displayed successfully
2	Multiple users access prediction page simultaneously	20	60	Application responds without failure
3	Continuous prediction requests	50	120	Stable performance with low response time

4	Invalid or missing input values	10	30	Error message displayed; application does not crash

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	0.82 sec	Pass	Fast prediction response
2	Response Time (Max)	< 5 seconds	2.34 sec	Pass	Within acceptable limit
3	Throughput (Req/sec)	> 20 req/sec	28 req/sec	Pass	Good request handling capacity
4	Error Rate	< 1%	0%	Pass	No failed requests during testing
5	CPU Utilization	< 80%	42%	Pass	Efficient CPU usage
6	Memory Utilization	< 80%	48%	Pass	Memory usage remained stable

Step 4: Observations & Analysis

Key Findings

- The Flask application handled prediction requests efficiently with low response times.
- The machine learning model generated accurate predictions without noticeable delays.
- The system remained stable during concurrent user requests.

Bottlenecks Identified

- Minor increase in response time when multiple requests were sent simultaneously.
- Large datasets may require additional optimization during preprocessing.

Optimization Steps Taken

- Saved the trained model using **Pickle** to avoid retraining during prediction.
- Optimized data preprocessing using Pandas.
- Reduced unnecessary computations by loading the model only once when the Flask application starts.
- Added input validation to prevent invalid requests.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):